
結果の取得と出力 その2

データの蓄積・保管と外部出力

Geant4 11.3.0 準拠

Geant4講習会資料: HEP/NP/Space 2025



KEK

大学共同利用機関法人

高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません
- 本資料の最新版は以下からダウンロード可能です
 - <http://geant4.kek.jp/lecture/>
- 本資料に関する問い合わせ先は以下です
 - Email: lecture-feedback@geant4.kek.jp

本講の目標

- スコアリングの全工程の実装の方法を学ぶ
- 全工程とは、
 1. スコアを取得し
 2. イベント単位でそれらをデータとして一時保管し、
 3. ランの間中、イベントごとのデータを必要に応じて集計し
 4. ランの終わりに出力する。
- 工程1は前の講義で扱った。工程2~4が本講義のテーマである。中心課題はデータの保管、集計、出力で、SensitiveDetectorに加えてEventAction, RunAction などが関わってくる。
 - 複数のクラスと複数のシングルトン・マネジャー(RunManagerとSDManager)が関与し、うまくデータの受け渡しをするには、考慮すべきことがいくつかある。
 - ・ クラスの関係がtight-coupleではGeant4の設計思想に反する
 - ・ 誰がデータを持つか、最終的にRunActionがそれにアクセスしてファイル出力するにはどうするか
 - ・ 独立したデータ専用のクラスを検討する
- まず、既成のCommand Based Scoring は限られた使用事例にしか使えないが、上述の全工程1~4を実装したものである。これを学ぶことから始めよう。

目次

1. コマンドベーススコアリング
 - ・ マクロコマンドの利用
2. スコアの取得・蓄積・保管
3. ヒットコレクションの取り扱い方
4. データの出力
5. LLMによるプログラミング補助

コマンドベーススコアリング

Command Based Scoring

できることは限られているが、スコアリングのための
C++コードを書かなくてよい

Geant4の備え付けスコアリングの一つである

- コマンドベーススコアラ (command-based scorers)
 - ジオメトリは用意せねばならない
 - スコアリングと出力のためのC++コードを書かなくてよい。用意されているコマンドを並べたマクロファイルを書くだけでよい。
 - できることは限られてそれで間に合うなら使いやすい。
 - ・ スコアはランごとに集計されて、イベントごとのスコアは扱えない。
 - ・ ジオメトリとは無関係に3次元ボクセルを定義でき、ボクセルごとにスコアが得られる。
 - ・ ボクセルの切り方とスコア量の選定の例
 - 直方体voxel分割をして、voxel毎の線量 doseDeposit を求めたい
 - 円筒形分割をしてエネルギー付与量energyDeposit の動径方向依存を求めたい
 - 種々の専用のコマンドが用意されている。
 - ・ メッシュの設定 (mass geometryとは無関係に自由にメッシュを切れる)
 - ・ スコアラの割り付け (今のところ17ヶ)
 - ・ フィルタ設定 (今のところ5ヶ) フィルターを満たすスコアだけを計上できる
 - ・ 結果の出力(CSVフォーマットに限られる)指定
 - メッシュの設定後、必要なだけのスコアラやフィルター付きスコアラをコマンドで指定できる。
 - ランが終わったら名前付きのCSVに結果が出力される。
 - お好みで、スコアの分布を可視化できる。
- スコアできる物理量と適応出来るフィルタは、医学分野や放射線防御などの応用例を想定して実装されている。汎用ではないが、自分の用途に利用出来ると分かれば、C++コード無しでスコアできる。

スコアラ ー覧 マニュアルの表8から一部抜粋

Name of primitive scorer	Description	Default unit	x-axis of 1-D histogram	y-axis of 1-D histogram
doseDeposit	deposited dose in the volume	Gy	dose per step in Gy	track weight
energyDeposit	deposited energy in the volume	MeV	eDep per step in MeV	track weight
trackLength	total track length in the volume (including both passing and terminated tracks)	mm	n/a	n/a
passageTrackLength	sum of track length in the volume for tracks that pass through the volume	mm	track length in mm	entry (unweighted)

最小限のC++コード: main()だけ

- この機能を使うには、main()の中で、G4RunManagerのインスタンスを作った直後に**G4ScoringManager**のインスタンスへのポインタを入手するだけ

```
#include "G4RunManager.hh"
#include "G4ScoringManager.hh"
int main(int argc, char** argv)
{
    G4RunManager* runManager = new G4RunManager{};
    G4ScoringManager::GetScoringManager();
    ...
    以下通常の必須初期化
```

スコアリングのために
C++で書くのは
これだけ

ハンズオン P09_Scoring1/**Water_Box_CBS**には上のmain.ccとTestBench/にはBraggピークのマクロの例"proton200MeV.mac"がある。コンパイルして実行するとCSVファイルを得る。

proton200MeV.mac マクロの内容

まずメッシュを切り、次にスコアを決め、閉める。ランを走らせた後でCSVファイルにダンプする。

```
$ cd ../TestBench  
$ less proton200MeV.mac
```

```
# define scoring mesh "boxMesh_1"  
#  
/score/create/boxMesh boxMesh_1  
# its size half lengths and number of bins in x, y and z  
/score/mesh/boxSize 15. 15. 15. cm  
/score/mesh/nBin 1 1 300  
#scores given name "eDep" and "dDep"  
/score/quantity/energyDeposit eDep  
/score/quantity/doseDeposit dDep  
/score/close  
/score/list  
# verbosity  
/control/verbose 2  
/run/verbose 1  
/tracking/verbose 0  
## particle  
/gun/particle proton  
/gun/position 0. 0. -15. cm  
/gun/direction 0. 0. 1.  
/gun/energy 200 MeV  
/run/beamOn 2000  
# Dump scorers to a file  
/score/dumpQuantityToFile boxMesh_1 dDep proton-dDep.csv  
/score/dumpQuantityToFile boxMesh_1 eDep proton-eDep.csv
```

スコア用メッシュ切り

“boxMesh_1” という名前の立方体で
寸法は15 x 15 x 15 cm (実寸の半分)
Bin数はx, y, z 方向に1, 1, 300とするの
で1ビンのz方向長さは 1mm となる

エネルギー付与(eDep)と線量付与(dDep)

Verbosity, 特にtrackingのお喋りを抑制する

WaterBoxの左端から右方向へ200MeVの
陽子を入射する。陽子の飛程を勘案してこ
のエネルギーを選んである。変えてみるの
もよいだろう。

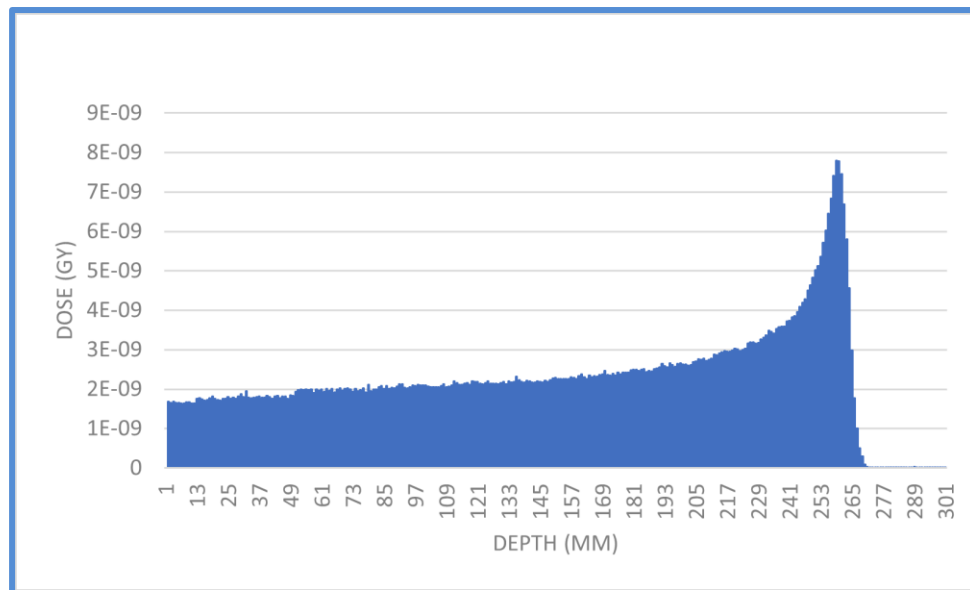
ランの後eDep, dDepをCSV形式で出力する

出力されたCSVファイルからヒストグラムを作成する

出力 proton-dDep.csv の冒頭部分確認

B3	v		x		fx		iY	
	A	B	C	この列			E	F
1	# mesh name: boxMesh_1							
2	# primitive scorer name: dDep							
3	# iX	iY	iZ	total(value) [Gy]		total(val^2)		entry
4	0	0	0	1.68E-09		4.33E-21		2000
5	0	0	1	1.66E-09		2.47E-21		1999
6	0	0	2	1.68E-09		3.53E-21		1999
7	0	0	3	1.65E-09		2.39E-21		1999
8	0	0	4	1.65E-09		2.45E-21		1998
9	0	0	5	1.64E-09		1.64E-21		1998
10	0	0	6	1.65E-09		1.64E-21		1997
11	0	0	7	1.67E-09		2.37E-21		1996
12	0	0	8	1.67E-09		2.86E-21		1995
13	0	0	9	1.65E-09		1.77E-21		1995
14	0	0	10	1.64E-09		1.70E-21		1995
15	0	0	11	1.75E-09		9.77E-21		1996
16	0	0	12	1.77E-09		8.84E-21		1996
17	0	0	13	1.74E-09		3.60E-21		1996

Excel での描画



H09ではgnuplotまたはjupyter labを使って
Bragg peakの描画をする

CBS 水ファントムの振り返りとスコアリングの課題

■ 何をしているか

1. ジオメトリは30 cm立方の水の立方体
2. Z方向に厚さ 1mm の薄い板に仮想的に分けて
3. 各板のなかでのエネルギーデポジットを求めランの間積算して
4. ランの終わりにCSVファイルに書く出す。

■ コマンドベーススコアラでは上の工程2－4をユーザに代わってやっている

■ 次は同じ結果を得るためにより一般的な方法を使うことを学ぼう

1. 先ず種々の物理量をG4Stepから取得することは学んだ
2. どの検出器でどうやって物理量を取得するか; SensitiveDetectorを学んだ
3. 本稿では物理量を蓄積・保管し、ランの終わりに書き出す方法および
4. データを解析してヒストグラムやプロットを得る方法を学ぶ

スコアの蓄積と保管

スコアの記憶領域について

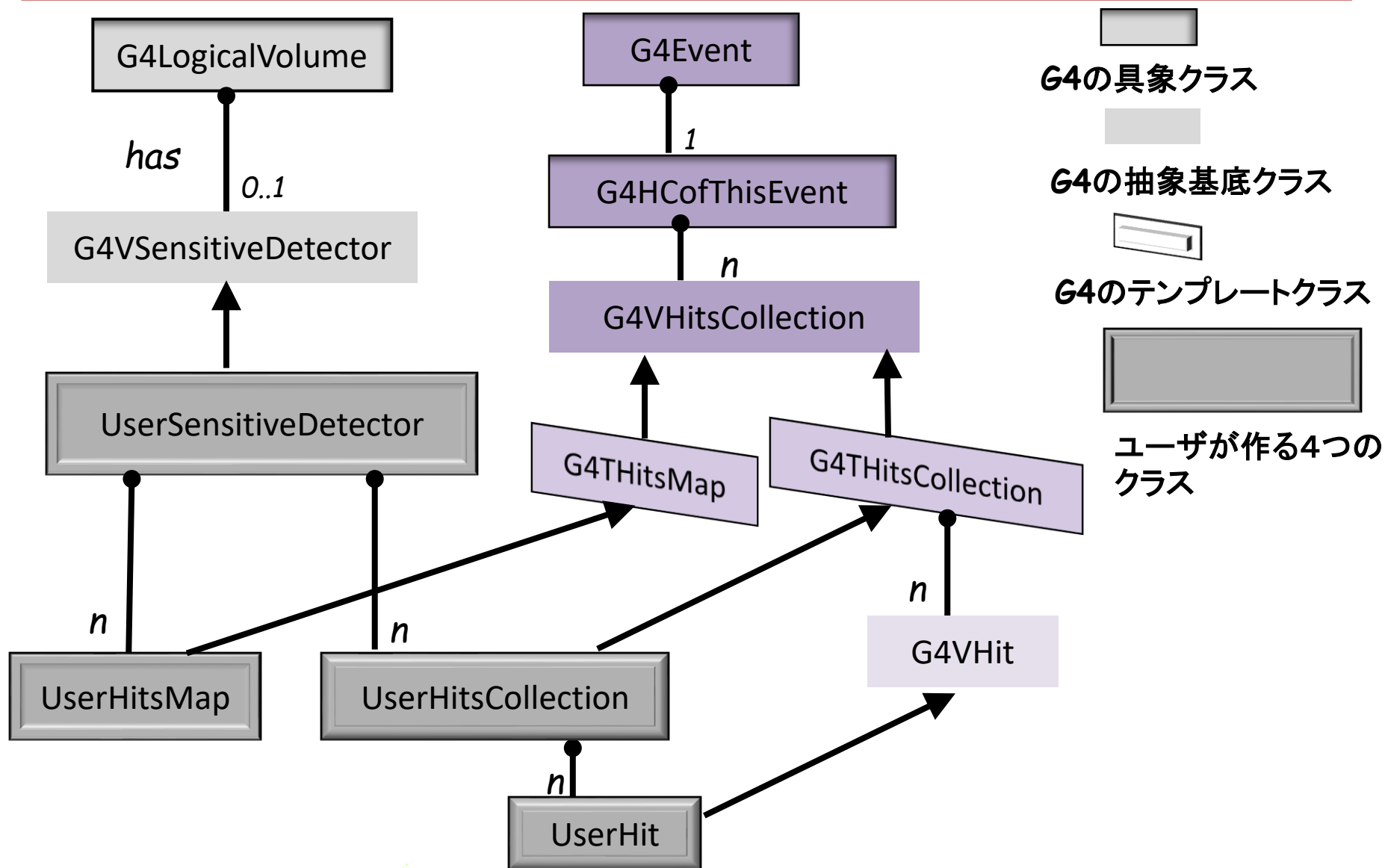
- OS上で実行中のプログラムをプロセスという。プロセスにOSが割り当てる仮想メモリー領域は
 - コード領域; OSがプログラムをロードして実行コードを置く不可侵領域
 - 静的領域; グローバル変数、static 変数などが置かれる記憶領域。
 - スタック領域; 局所変数、引数などを置く可変長の領域で、使わなくなったら自動的に解放される
 - ヒープ領域; ユーザが動的に確保し、解放できる領域。C++ではオブジェクトの **new** で使えるようになり、**delete[]** で解放する
- Geant4のスコアの格納には基本的に heap ヒープ領域を使う。C++ 標準のSTL (Standard Template Library) **std::vector**, **std::map**などを使うと大きなヒープ領域を確保できる。また、iteratorやさまざまな関数を利用できる。
 - std::vector は一番よくつかわれているが、不用意に使うと実行時にヒープ領域オーバーフローエラーを起こすことがある。
 - 例えば、一定領域を指すポインターを要素とするベクトルではベクトルを delete しても指していた領域はそのまま残り、いわゆるメモリーリーク (漏れ)が発生する恐れがある
 - ハンズオンでは、std::map, std::tuple std::vectorを扱う。

データのスコープと保持時間

1. Main プログラムではRunManager, Geometry, Physics, UserActionInitializationがインスタンス化された後に
2. **G4RunManager**が**Initialize** される。
 1. ここでGeometry::ConstructSDandField()
 2. **G4SDManager**が立ち上がって、SensitiveDetectorを割り当てる
3. RunManagerがbeamOnすると
 1. BeginOfRunAction
 1. GeneratePrimaries
 2. BeginOfEventAction
 1. ProcessHits 繰り返し Hit Data発生
 3. EndOfEventAction Hit Dataのaggregation
 2. EndOfRunAction a acumulated data 出力

Sensitive Detectorで発生したHitデータをどこで蓄えて誰がいつ引き出すか

SensitiveDetectorとスコアデータの公式クラス図



Geant4 提供のデータ蓄積機能をちょっとだけ見ておこう

- クラス図では **G4HCofThisEvent** (Hit Collection of This Event)の作り方を定めている
 1. スコアを格納する自分の”UserHit” は**G4VHIT**を継承して実装する。
 2. “UserHit”を集めた”UserHitsCollection”を作る。これはG4VHitsCollection継承するテンプレートクラス G4THitsCollection を継承し実装する。
 3. **UserEventAction**はG4Event の次の関数でそのポインタを得ることができる
 - inline G4HCofThisEvent* GetHCofThisEvent() const
 4. イベントで得られたヒットコレクションをランを通してどのように**集約・集計**するかはユーザの仕事となる
 5. ヒープ領域を確保するにはアロケータを使う。
 6. 小規模なシミュレーションには複雑すぎて分かりにくい。
 7. 集約・集計について
 - Geant4/examples/ にはこれを使った例が多数見当たるが、ランを通しての集計は、多くの場合シングルトン解析マネージャーである**G4AnalysisManager**に投げている。

データの発生、蓄積、保存の流れとG4AnalysisManager

- 実験では測定器からのデータを保存し、2次3次とフィルターをかけてデータ圧縮ファイルを作成する過程とそれを解析する過程は厳然と分かれている。ひとつのデータ圧縮ファイルを多数のチームが独立に解析するのは常識である。つまり**データの発生・保存はデータの解析とは独立**している。
- これとは違って、G4AnalysisManagerはデータの保存、ヒストグラムなどの解析への集積、ランの終わりにROOTやCSVへの出力をすべてまとめて取り扱う。作成するヒストグラム(そのビン数なども)やプロットなどはランのコンストラクタでG4AnalysisManagerに登録しておく必要がある。
 - ・ スコアが発生してヒストなどにFillなどの関数を使用するときにはその都度G4AnalysisManagerのシングルトンインスタンスを手繰って関数を使用する。ランの終わりにもシングルトンを手繰ってファイルへ出力する。 Ntuple 出力だけが解析の独立
 - アナリシスマネージャは、スコアデータまたはそれらのヒットコレクションをHist1D, Hist2Dや Ntuple などのオブジェクトで蓄積し、ランの終わりにROOTやCSVファイル形式で出力できる
 - ・ examples/basic, advanced, extended の多くは解析ツールROOT形式」出力して可視化するためにこれを使っている。
 - ・ アナリシスマネージャはCSV, XML 形式のファイル出力も選択できる。
- データ発生・保存とデータ解析とを分離するには、目的に合わせてデータ構造を設計し、スコアをそれに記入・保管してファイルに出力する方法が考えられる。

スコアデータの蓄積・保管とファイル出力を自前でやる選択

- 簡単な場合にはHCoFThisEventを使わずに `std::map`, `std::tuple`, `std::vector` などを使って自前でデータ構造を作成してスコアの蓄積・保管からファイル出力を行うことが分かりやすい。。ハンズオンではこの手法を使う。
- “大規模”実験目的でない多くのユーザには、アナリシスマネージャーに頼ることなく、`std::tuple`, `std::map`, `std::vector` など(およびその組み合わせ)を目的に合わせて使い、適当なファイル形式で出力するのが良い。解析のやり方や解析ツールを自分で決められる。
- そこで、比較的簡単なスコアリングでは、解析マネージャを使わず、自前でスコアリングデータを蓄積・保管してファイル出力するやり方の課題を考える。
 - ・ データの蓄積と保管を誰が担当するか
 - ・ 蓄積したヒットデータをファイルに出力するのが適当だろう。これは `UserRunAction` の役割とする。
 - ・ 出力ファイル形式はCSVとする。必要ならXMLによるSerialization も検討の価値がある。
- CSVファイルの解析と可視化には、従来の `gnuplot` よりもはるかに使いやすく機能も豊富なJupyter lab などデータサイエンス分野で使われるものが使える。

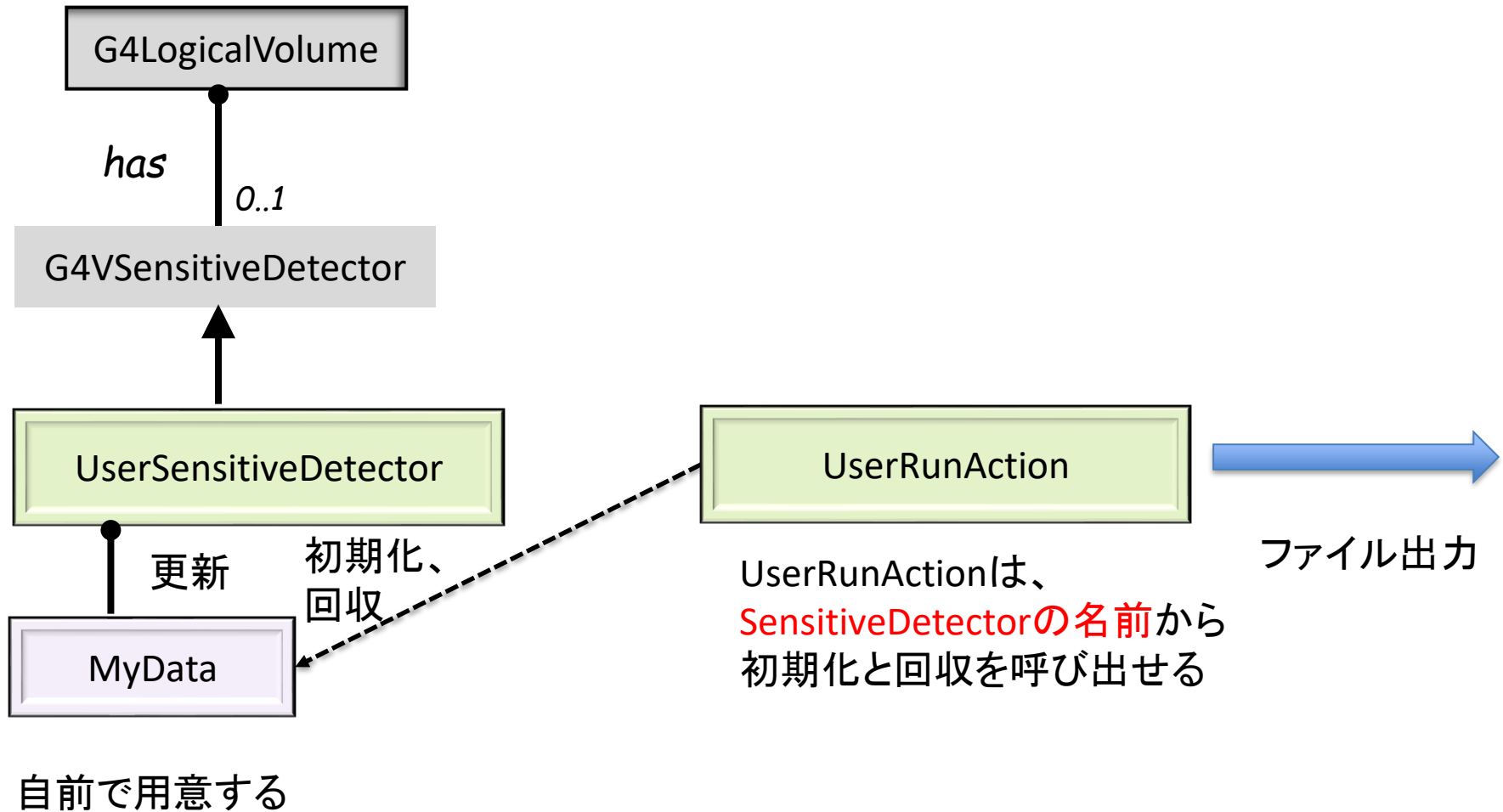
ヒットコレクションの取り扱い方

スコアデータをどう扱うか、いくつかの案

- 全体: スコアを取得するのはSensitiveDetectorでスコアを出力するのはRunAction
- スコアデータを誰が持つか。
- S案) SensitiveDetectorに持たせその更新をやらせる。RunActionはSensitiveDetectorのインスタンスをG4SDManagerから得てデータを引き上げて出力する
 - ✓ SensitiveDetectorとUserRunActionが密に絡んでくる。
- D案) 独立したデータストア専用のクラスを用意する。
 - ✓ データストアのデータ構造は他のクラスに依らず設計できる。
 - ✓ データストアの参照を関連するクラスが使えるようにする
- A案) G4AnalysisManager に任せる
 - ✓ シングルトンなので、どのクラスからでもアクセスできる
 - ✓ RunAction のコンストラクタでスコアのヒストグラムなどを予め決めておいて

以下では、G4AnalysisManagerは使わず、S案とD案とを少し深掘しよう

S案: SensitiveDetectorにデータストアを持たせる



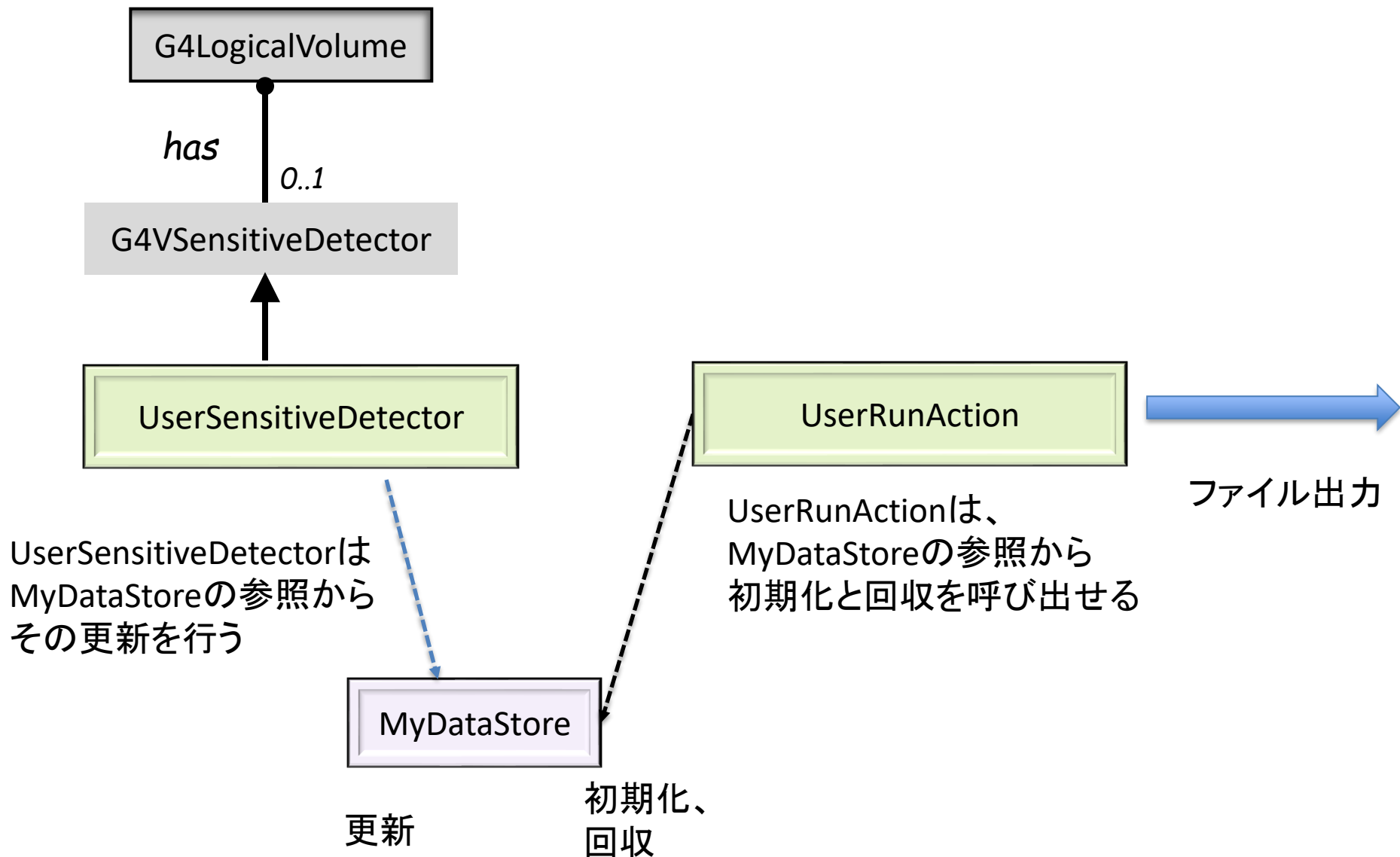
SensitiveDetectorに自前のデータ構造を持たせる案

- SensitiveDetectorの数が少ないときには、各SensitiveDetectorクラスのデータメンバーとしてデータ構造をもたせると単純である。
- **G4SDManager** はすべてのSensitiveDetectorやG4HitCollectionのインスタンスへのアクセス方法を提供している。
 - SensitiveDetectorはprivate なMyDataを持つ。
 - たとえば SensitiveDetector からのHitをstd::map に入れるとしよう。ランの間中に発生するHitを集めてstd::vectorに格納する
 - EndOfRunActionでG4SDManager::G4SDMpointerを呼び出しその**FindSensitiveDetector("name")**を使ってUserRunActionは SensitiveDetector のインスタンスへアクセスできる。
- 特色と課題
 - スコアリングデータはSensitiveDetectorと密にカップルしていて責任範囲が明確に決まっている。
 - データストアの数が少なくその扱いもよくわかっているので、RunActionが無駄に複雑になることはない。

独立したデータストア専用のクラスを用意する方法

- 本来Runは測定器を準備し、ビームを準備したら、すべての実験条件が変更されないように実験エリアにインターロックをかけて立ち入り禁止にした後ビームオンし、一定の数のイベントが溜まったらビームオフとする役割を果たすだけのものである。検出器で得られたデータはレジスタなどに一時的に保存され、ランの終わりには取り出され磁気媒体などに書き出される。
- そこで、検出器にもランにも結び付かない**独立したデータ保管庫を用意**すれば、データの保存や取り出し操作などを他のクラスに合わせて作り変える必要はなくなる。また、データの所有者とその寿命を分かりやすく制御できるようになる。
- ハンズオン **P09_Scoring1/Bragg-MyDataStore** と **P10/Pixel_Four_Output** は専用のデータストアクラスを作る例である。この手法では
 - 各クラスは自分自身の良く定義されたものにてできる
 - クラスの再利用や変更がやさしい
 - クラスの定義と実装が自然である
- プライベートデータストアが持つべきメソッド
 1. データストアの初期化とクリーンアップ機能
 2. データupdate/Fill更新機能
 3. データretrieval/Get回収機能

D案: 独立したデータストア

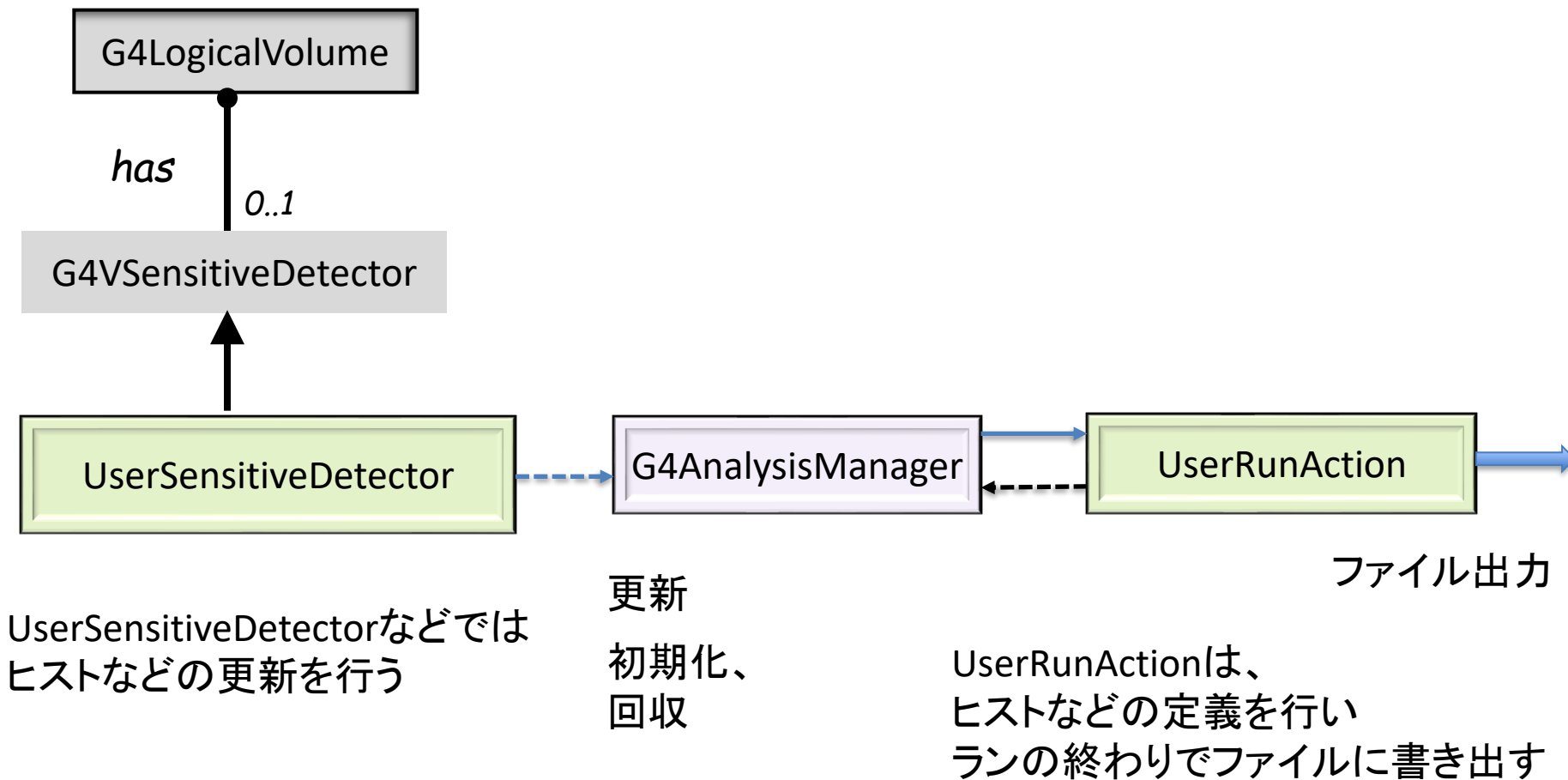


データストアへの参照を渡すべきクラス

- Stand-aloneのデータ保管庫クラスを作成し、そのインスタンスを明示的にnewし、それへの参照を関係するクラスに渡す。
- 段取り: DataStoreインスタンスを関係するクラスに渡す順序
 - メインプログラムでRunManagerよりも前にインスタンスを作りその参照を以下の順序でわたす。
 1. Geometry はConstruct()のなかでSensitiveDetectorのインスタンスをつくり、参照を受け取る
 2. UserActionInitialization はUserRunActionに参照を渡す

P09/Bragg_DataStore およびP10/Pixel_Four_Outout
は独立したデータストアを使う例であるが、実装は少しだけ違っている

A案: G4AnalysisManager



解析マネージャを使う RunAction での準備と始末

- シングルトンである解析マネージャG4AnalysisManager を使いたいクラスはその都度インスタンスを呼び出して使う。

```
G4AnalysisManager* manager = G4AnalysisManager::Instance();  
manager-> Methods();
```

- ランを始める前の準備作業には二つある
 1. RunAction のコンストラクターではヒストグラムやスキャッタプロットを定義する
 - AnalysisManager のインスタンスを呼び、
 - ヒストグラムの定義 ; manager -> CreateH1() など
 - Ntuple の定義 ; manager -> CreateNtuple()
 2. ランを始める前にBeginOfRunActionで行うこと
 - AnalysisManager のインスタンスを呼び、
 - 出力ファイルのオープン manager -> OpenFile("runName");
- ランの終わりにEndOfRunActionですること
 - AnalysisManager のsingleton インスタンスを呼び、
 - ファイルへの書き出し manager -> Write();
 - ファイルのクローズ manager -> CloseFile();

ランマネジャー ステップ・イベント毎にスコアをFillする作業

- ヒストグラムなどのFill()を行う場所

1. SteppingActionの中
2. SensitiveDetector::ProcessHits()やEndOfEvent() のなか
3. EventAction::EndOfEvent のなか

- Fill メソッド

- ✓ 1次元、2次元、3次元のHistograms; H1, H2, H3
 - スコアを記入する; manager -> FillHi() i=1..3
- ✓ Ntuple
 - 任意の大きさの2次元行列
 - 各行にはそのイベントで得られたスコアを格納する
 - Ntupleのセルのデータ形式に合わせて記入する(**S**, **I**, **D**)
 - manager -> FillNtuple**D**Column(ntupleID, columnID, value);

スコアの外部出力

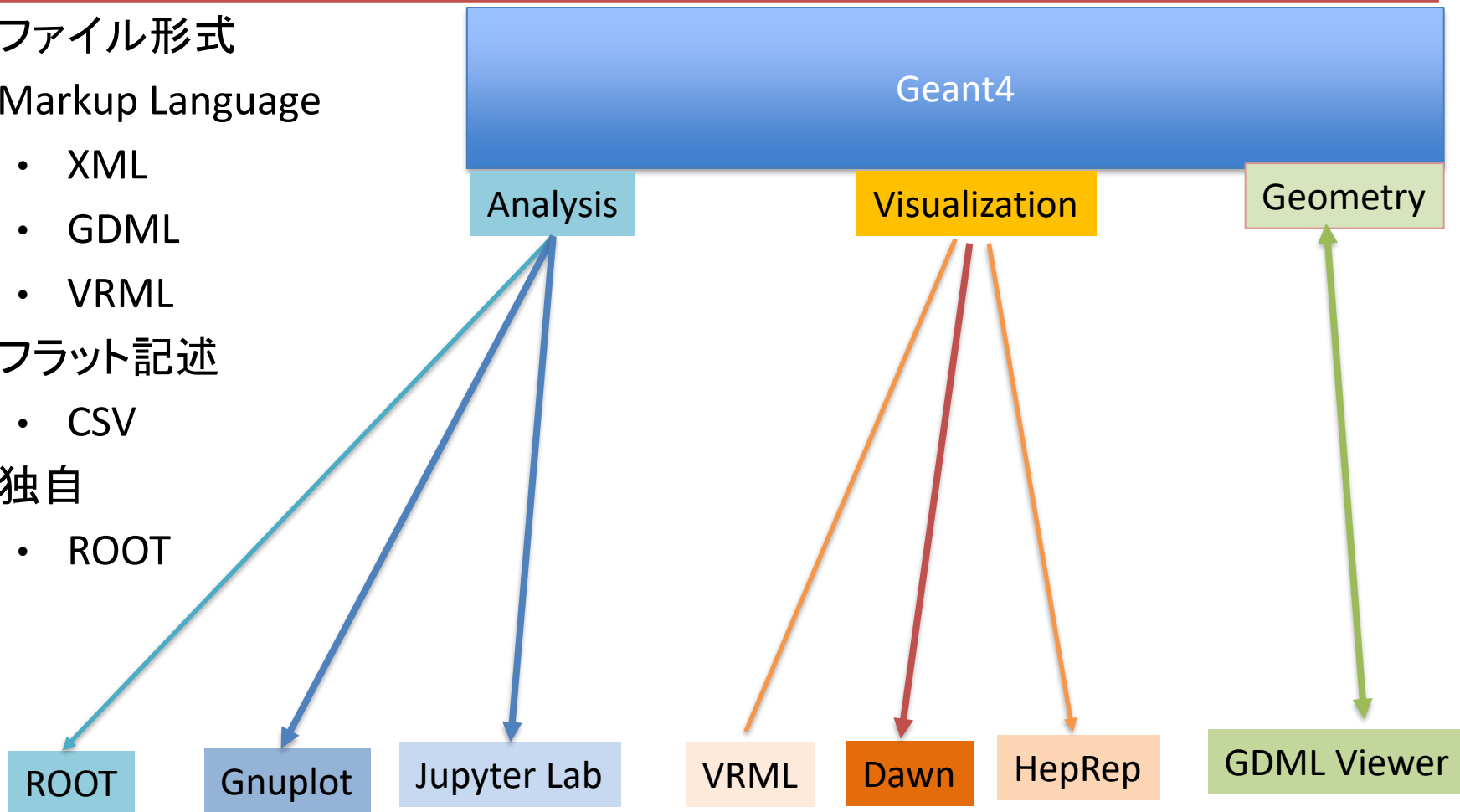
Object Persistency : オブジェクトの永続性 と出力

- C++ は Object Persistency の機能を持っていない。Geant4もそう。
 - Heap 領域に作られたすべてのオブジェクトはプロセスが終了したときに消えてしまう。ヒット情報もその運命にある。
 - 永続オブジェクトはプロセスが終了しても存続し、別のプロセスからアクセスできるものである。
 - Geant4 に汎用の永続オブジェクトを持たせる試みは実現できていない。出発当初オブジェクトデータベースを利用する試みはあったが、Geant4がそのような外部ソフトに依存することになると、Geant4の命運がそのソフトによって決められてしまうことになる。
- Geant4 プロセスの中で存在するオブジェクトをファイルに保存するために、Geant4の各カテゴリーの開発者の考えに応じていろいろな方策が提供されている。
 - Analysis カテゴリー xml, root, csv,
 - 可視化カテゴリー vrml, .prim, csv, xml
 - ジオメトリカテゴリー gdml

Geant4 ファイル出力 (参考までに)

■ ファイル形式

- Markup Language
 - XML
 - GDML
 - VRML
- フラット記述
 - CSV
- 独自
 - ROOT



Geant4 G4AnalysisManager の出力ファイル形式

- HEPではいろいろの解析ツールが使われているが、ヒストグラムを作ったり、記入したり、ファイルに保存するなどのスコアを解析する操作は共通である。

- このような操作を記述するGeant4アプリのC++ソースコードは解析ツールには依存してはならないのを建前としていて、解析ツールに合わせた形式のファイルを出力する

出力ファイル形式	解析ソフト	特長
CSV	表計算, GNUPLOT, jupyter lab	二次元の表形式
ROOT	ROOT	固有形式
XML	JAS, OpenScientist, AIDA, Python	階層的データ構造

□ analysis カテゴリーは

- ✓ 異なる解析ツールに依存しない統一的なインターフェイスを提供する
- ✓ ユーザコードでは、出力形式に関係なく解析マネージャへのポインタを参照するだけで良い。
- ✓ マルチスレッドに対応する

Python による解析

Pythonは最強のデータ解析ツールの一つ

- 統計解析、フィッティング、推計、可視化のために役立つもの。赤字のものはP10_Pixel_Four_Outout の解析で使う。

- **numpy**: 数値計算
- **pandas**: CSVファイルの読み込みやデータ解析
- **matplotlib**: グラフ描画
- **seaborn**: 統計的データ可視化
- **scipy**: 最小二乗法フィットや統計解析
- **scikit-learn**: 機械学習や回帰分析
- **statsmodels**: 統計モデルの推定

■ CSVファイルの読み込み

```
import pandas as pd
# CSVファイルを読み込む
df = pd.read_csv("data.csv")
```

■ xmlファイルの読み込み

```
import xml.etree.ElementTree as ET
#xmlファイルを読んでrootを得る
tree = ET.parse('Data_file.xml')
root = tree.getroot()
```

JupyterLabの使い勝手

- Python や R の豊富なライブラリーを使える
- インターラクティブにコードの作成、実行、表示、修正ができるので効率的に解析作業ができる
- ノートブックに記述したコードを実行し、結果もまたノート内に表示されるので共有が簡単である。
- コード、実行結果、説明文を一つのノートブックにまとめられるので、チームでの作業に役立つ。
- ブラウザで実行環境を提供しているのでプラットフォームに独立なデータ解析環境を使える。
- Vscode ではエディタ、Python実行環境、Copilotを提供

P10/Pixel_Four_Outputにコードと結果まで含んだノートの例がある。
次の2ページはそのコードの一部である

Markdown cell とコードの例 H10 のPixel_Four_Output

Markdown

タイトル

セクション

サブセクション

普通のテキストです。

****太字****や***斜体****の使用も可能です。

リスト項目:

- 項目1
- 項目2

数式の例:

$E = mc^2$

LaTeXも使える

```
P10_Randomized.ipynb x +
+ ✂ 📄 ▶ ■ ↺ ⏪ Code ▼ Notebook 📄 🐍 Pyt

import libraries

• pandas
• numpy
• matplotlib's plt
• scipy's optimize

Read CSV file and store it in df

• pixel_replica regroup df according to pixel number

define functions

• calculate_stats gets mean and standard deviation of x and y coordinates at given pixel
• plot_histogram and plot_scatter using world coordinates
• fit_circle Least Square Fitting to a circular trajectory
• convert_df2 to randomize copy number using its center position and width with uniform rbdm number. Then, same fit_circle as above

[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares

def calculate_stats(data, column):
    mean_value = data[column].mean()
    std_value = data[column].std()
    return mean_value, std_value

def plot_histogram(data, column, bins, xlabel, ylabel, title, filename):
    plt.hist(data[column], bins=bins)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    plt.savefig(filename)
    plt.show()

def plot_scatter(data, x_column, y_column, xlabel, ylabel, title, filename):
    plt.scatter(data[x_column], data[y_column], c="red", s=1, marker=".")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
```

関数の例 P10/Pixel_Four_Output で使用

■ ヒストグラムとプロットの関数定義

- matplotlib.pyplot をimportして plt と別名を付ける
 - hist, scatter, xlabel, ylabel, title, savefig, show を使う。Pyplotには他にもたくさんの機能がある。

```
import matplotlib.pyplot as plt
```

```
def plot_histogram(data, column, bins, xlabel, ylabel, title, filename):  
    plt.hist(data[column], bins=bins)  
    plt.xlabel(xlabel)  
    plt.ylabel(ylabel)  
    plt.title(title)  
    plt.savefig(filename)  
    plt.show()
```

```
def plot_scatter(data, x_column, y_column, xlabel, ylabel, title, filename):  
    plt.scatter(data[x_column], data[y_column], c="red", s=1, marker=".")  
    plt.xlabel(xlabel)  
    plt.ylabel(ylabel)  
    plt.title(title)  
    plt.savefig(filename)  
    plt.show()
```

最小二乗法フィットの例

- 与えられた点 (x, y) の集合に対して最小二乗法を用いて円をフィットして、その中心座標 $(c[0], c[1])$ 、半径、および半径の誤差を計算する関数。非線形関数でも使える。

```
import numpy as np
from scipy.optimize import least_squares

def fit_circle(x, y):
    # 円の中心(c[0], c[1])と各点(x, y) から円の半径を求める
    def calc_radius(c):
        return np.sqrt((x - c[0])**2 + (y - c[1])**2)
    # 全データ点座標から計算した半径の平均値と点ごとの差。これを最小とする
    def fun(c):
        ri = calc_radius(c)
        return ri - ri.mean()
    # 初期推定値
    center_estimate = np.mean(x), np.mean(y)
    # LSQFit
    result = least_squares(fun, center_estimate)
    center = result.x
    radius = calc_radius(center).mean()
    residuals = calc_radius(center) - radius
    radius_error = np.sqrt(np.sum(residuals**2) / len(residuals))
    return center, radius, radius_error
```

Visual Studio code on Windows 11

Python 実行環境も使える
右の例はPixel_Four_Outputの
解析例

さらにLLMも同じウィンドウで使える

The screenshot displays the Visual Studio Code editor with a Python script named `P10_Radius.ipynb` open. The script includes code for plotting histograms and scatter plots for coordinate Y. The output of the script is visible in the console, showing statistical data for pixels 1000, 1001, 1002, and 1003, including mean, sigma, and circle center coordinates. A histogram plot titled "Coord Y at Pixel #1 to #4" is also shown, displaying frequency versus Y (mm). The right sidebar features the "Welcome to Copilot" panel, indicating that Copilot is your AI pair programmer and offering options to sign in to GitHub.

```
# Plot histogram of coordinate Y including all pixels
plot_histogram(df, 5, bins=400, xlabel="Y (mm)", ylabel="Frequency", title="Coord Y at Pixel #1 to #4")
plot_histogram(df, 2, bins=50, xlabel="copyNo_Y", ylabel="Frequency", title="copyNo_Y")
# Plot scatter plots for each pixel replica
for pixel_id, data in pixel_replicas.items():
    plot_scatter(data, 4, 5, xlabel="X (mm)", ylabel="Y (mm)", title=f"X at Pixel {pixel_id}")
```

[1] ✓ 2.9s Python

Y at Pixel 1000, mean = 0.1559, sigma = 0.0013
X at Pixel 1000, mean = 0.0001, sigma = 0.0014
Y at Pixel 1001, mean = 1.4121, sigma = 0.0444
X at Pixel 1001, mean = 0.0019, sigma = 0.0482
Y at Pixel 1002, mean = 3.9304, sigma = 0.1033
X at Pixel 1002, mean = 0.0043, sigma = 0.1069
Y at Pixel 1003, mean = 7.7077, sigma = 0.1797
X at Pixel 1003, mean = 0.0058, sigma = 0.1770
Circle Center = (1991.3854, -100.1693), Radius = 1991.3877, Radius Error = 0

Coord Y at Pixel #1 to #4

Frequency

Y (mm)

copyNo of Y

Welcome to Copilot

Copilot is your AI pair programmer.

Code faster with Completions

Build features with Copilot Edits

Explore your codebase with Chat

We now offer Copilot for free.

Signing in to GitHub...

By continuing, you agree to the Terms and Privacy Policy.

Copilot Free and Pro may show public code suggestions and we may use your data for product improvement. You can change these settings at any time.

Signing in to github.com...

Source: GitHub Authentication

Cancel

LLM AIのプログラミング補助

P09Scoring1/Braff-DataStore を手伝わせた

Gemini, ChatGPT, Copilot 無料版を使った

Gemini 2.0 Flash は、Google が開発した最新の生成 AI モデルで、Pixel 9 シリーズに搭載されています。

Gemini 2.0 Flash は、テキスト、画像、音声、動画など様々な形式のデータを処理できるマルチモーダル機能を備えており、より高度な自然言語処理や画像認識などのタスクを実行できます。

はい、今使っているのはGPT-4oだよ。「o」は「omni(オムニ)」の意味で、以前のGPT-4よりも高速で高性能になっているよ。

私はMicrosoftによって開発されたAIコンパニオン「Copilot」ですので、GitHubとの直接的な関係はありません。ただし、MicrosoftはGitHubを所有しており、GitHub Copilotという別のAIツールも存在します。そのツールは特に開発者向けに、コードの補完や提案を行う機能を持っています。

- ハンズオン P09/Bragg_DataStore
- 詳細な仕様を記述したプロンプトを与えすべてのコードを一括提出させた。
- 生成されたコードをそのままコンパイルしたらエラーが出た。それをカット＆ペーストして見せたら修正したコードを提出した。これを数回繰り返して動くコードができた。

著作権について

Copilot:

素晴らしいご指摘ありがとうございます！確かに、私が生成するC++コードは完全ではなく、専門家の視点での訂正や改良が必要な場合があります。

私が生成するコードはあなたの参考や支援のためのものであり、その著作権を主張することはありません。最終的にあなたが訂正したり改良したコードは、完全にあなたのものです。

私はツールとして、あなたの作業をサポートする役割を果たしています。

ChatGPT:

あなたがAIのコードを訂正し、独自の改良を加えた場合、その修正・改良部分にはあなたの著作権が発生すると考えられる。

一方、AIが生成した元のコードには著作権がないので、誰でも自由に利用できる。ただし、最終的な判断は各国の著作権法によるので、法律の専門家の意見を聞くのが確実だね。

Copilotにコード生成をさせる指示文

Let's create a simple but illustrative example code of Geant4 application.
Objective is to get depth-energy deposit relation, so called **Bragg peak** for incident protons into a water phantom.

let's go step by step to create definition and implementation files.

1. Geometry class Geomtry.hh and Geometry.cc

- world is a cube of 1 m filled with air.
- a water phantom is a cube of 30*cm is filled with water.
- It is sliced into 300 plates of 1*mm thickness using **G4PVReplica** in Z direction
- In Geometry::Construct method the logical volume of the plate is assigned to the Sensitive detector with name "**WaterPhantom**".
- the reference to "**DataStore**" must be known here.

2. **DataStore** class

- it has a private store for energy deposit at given copy number plate.

We use private:

```
std::map<G4int copyNo, G4double energy>myMap
```

- The class must have public methods ;
InitializeMap(myMap) to clear the map at the RunAction::BeginOfRun,
- and public: **GetMap** which is used at the RunAction::EndOfRun.
- **FillMap**() method which is used by the SensitiveDetector to fill score

つづき

3. SensitiveDetector class SensitiveDetector.hh, SensitiveDetector.cc

- SensitiveDetector::Initialize does nothing
- SensitiveDetector::ProcessHits function must accumulate energy deposit of G4Step at this copyNo.

It gets the copyNo of the plate and total energy deposit therein.

- SensitiveDetector::EndOfEvent does nothing

4. RunAction class RunAction.hh RunAction.cc

- it must be capable of getting the reference to DataStore
- BeginOfRunAction do Initialize DataStore's myMap.
- EndOfRunAction use DataStore's GetMap method and output the Map to a file in CSV format "copyNo, energy" and output to a CSV file "Bragg.csv"

5. UserActionInitialization class must be properly initialized.

it must be capable of getting the reference to DataStore

6. PrimaryGenerator class must be properly made.

7. Main program: Application_Main.cc

- include all necessary files
- be careful of the order of initialization
- **DataStore is instantiated** here and its reference must be passed to relevant classes.
- RunManager is "Serial" mode and be initialized at appropriate timing.
- interactive mode with visualization
- proper physics list must be used
- macro file "GlobalSetup.mac" must be called at start up time.

Fin